2024 IEEE Pacific Rim Conference on Communications, Computers and Signal Processing (PACRIM) | 979-8-3503-6231-2/24/$31.00 ©2024 IEEE | DOI: 10.1109/PACRIM61180.2024.10690198

Fairness Improvement of Different TCP Congestion Control Algorithms using P4 Programmable Data Plane

Koshi Ono[1], Daiki Nobayashi[1], Takeshi Ikenaga[1]
[1]Kyushu Institute of Technology, Fukuoka, Japan
e-mail: ono.koshi266@mail.kyutech.jp, {nova, ike}@ecs.kyutech.ac.jp

***Abstract*—A wide variety of congestion control algorithms have been developed for the transport protocol of TCP/IP. A related study revealed that the communication performance becomes unfair depending on the buffer length of switches or routers in the end-to-end path when communication flows using CUBIC TCP and Bottleneck Bandwidth and Round-trip propagation time (BBR) compete with each other. Therefore, we propose the fairness improvement method between different types of congestion control in the programmable network by using P4 programming. The proposed method provides multiple queues to each congestion control in the switches or the routers for relaying. The proposed method then assigns incoming TCP flows to those queues for each congestion control algorithm. The congestion control algorithm is determined by P4, which implements a programmable data plane. Three types of packet scheduling are used to ensure fairness when sending packets. We evaluate the effectiveness of the proposed method using the network emulator "mininet" and the communication performance when the queue length of the relaying switch.**

***Index Terms*—Programmable data plane, P4, Congestion control, BBR, CUBIC**

I. Introduction

With the continuous Internet technology, several congestion control algorithms have been developed for the transport protocol of TCP/IP. Based on a study of web access, CUBIC TCP [1] represents 36% of usage while Bottleneck Bandwidth and Round-trip propagation time (BBR) [2] represent 22% [3]. These two algorithms are the primary congestion control algorithms. CUBIC is a congestion control algorithm that determines the congestion window size based on packet loss, using a loss-based approach. BBR is a new congestion control algorithm developed by Google in 2016. BBR operates with inflight packets close to the optimal point proposed by Leonard Kleinrock [4]. When CUBIC and BBR compete with each other, BBR occupies bandwidth when the buffer size of the bottleneck link is small due to the difference in the index for determining congestion. Conversely, when the buffer size of the bottleneck link is large, CUBIC occupies the bandwidth [5]. Therefore, to ensure fair network utilization, there's a need to solve the throughput unfairness between CUBIC and BBR. This study focuses on the programmable data plane as a means of improving throughput fairness. The functionality of network forwarding equipment can be divided into a control plane, which determines routing rules, and a data plane, which actually forwards data. Even though SDN (Software Defined Networking) enables programming of the control plane, SDN cannot modify the data plane. Therefore, packet forwarding has been performed using packet processing functions defined by the forwarding equipment developer. Programmable data planes allow programmers to customize packet processing functions. By using a programmable data plane, it's possible to achieve flexible control instead of relying on fixed packet processing functions, thus improving throughput fairness at relay nodes. In this paper, we propose a method to improve fairness by using a programmable data plane. In the proposed method, relay nodes determine the congestion control algorithm of received packets and assign them to buffers corresponding to each congestion control algorithm. This method avoids contention between different congestion control algorithms in the buffer. We evaluate the effectiveness of the proposed method by emulation.

The remainder of this paper is as follows. Section 2 describes related research. Section 3 describes the proposed method. Section 4 presents performance evaluation experiments using emulation. Finally, we conclude the paper in Section 5.

II. Related work

A. Throughput fairness in the case of CUBIC and BBR competition

When CUBIC and BBR share a bottleneck link, the throughput is unequal depending on the buffer size. When the buffer size of the bottleneck link is small, BBR outperforms CUBIC in throughput, and conversely, when the buffer size of the bottleneck link is large, CUBIC outperforms BBR in throughput [5]. When the buffer size of the bottleneck link is small, the loss-based CUBIC reduces the congestion window size due to frequent packet loss. In contrast, the congestion window size of BBR does not change with packet loss, but is determined by the bandwidth delay product. Therefore, when the buffer size of the bottleneck link is small, the transmission rate of CUBIC is lower than that of BBR. As a result, BBR outperforms CUBIC throughput. When the buffer size of the bottleneck link is large, CUBIC increases its sending rate until packet loss occurs. In contrast, BBR is expected to increase the congestion window size due to higher buffering delay. A possible cause of

Authorized licensed use limited to: INDIAN INSTITUTE OF INFORMATION TECHNOLOGY. Downloaded on January 31,2026 at 16:35:05 UTC from IEEE Xplore. Restrictions apply.

BBR performance degradation is that the buffers are occupied by CUBIC, which prevents BBR from operating correctly at its optimal operating point. Therefore, when the buffer size of the bottleneck link is large, the transmission rate of CUBIC is higher than that of the BBR. As a result, CUBIC outperforms the throughput of the BBR. When the buffer size of the bottleneck link is of a certain size, CUBIC and BBR have the same throughput as each other, and it has been confirmed that there are repeated intervals where CUBIC is dominant and BBR is dominant.

B. Study on throughput fairness

Various studies have been conducted on throughput fairness in the contention of different congestion controls. Katumata et al. [6] measure RTT using ICMP in user space, and improve fairness by passing accurate RTT values to BBR.

Y.-J. Song et al. [7] [8] proposed the BBR-CWS (BBR Congestion Window Scaling) scheme to improve the inter-protocol fairness of BBR. They confirmed that BBR-CWS and CUBIC improved fairness and reduced the number of packet retransmissions compared to BBR. The performance of BBRv2 is also evaluated. The results show that BBRv2 improves fairness in small buffers below 1 BDP and that BBRv2 improves fairness and excessive packet loss when sharing bottleneck links. However, there are still some problems such as RTT unfairness and loss-based congestion control.

Ishikura et al. [9] focus on the problem that when BBR conflicts with loss-based congestion control, BBR misestimates RTprop due to queues that regularly exist in the buffers of bottleneck links, resulting in worsening network congestion. They improve this problem by estimating and adjusting RTprop using priority packets.

Ranysha Ware et al. [10] note that when TCP BBR competes with loss-based TCP, TCP BBR occupies the bandwidth regardless of the number of loss-based flows. In this study, one flow of TCP BBR competes with 16flows of CUBIC TCP, indicating that TCP BBR occupies the bandwidth.

Satoshi Utsumi et al. [11] devised a model for estimating the average packet transmission rate of CUBIC when CUBIC and BBR compete on a bottleneck link and improved fairness.

The fairness improvement methods introduced in this section focus on the control of the sending terminal. Improvement methods at the sending terminal may have problems such as the need to change the control of each sending terminal and an increase in the load on the sending terminal. In this study, we evaluate the effectiveness of CUBIC and BBR in improving fairness at the relay node in the case of contention.

III. Proposed method

As a means of improving throughput fairness when CUBIC and BBR contend, we propose a method of storing CUBIC and BBR packets in different buffers using the programmable data plane. TheProgrammable data plane can be programmed in P4 language [12] to customize packet processing functions. The programmable data plane enables flexible control of data planes that have been using fixed packet processing functions.

In trditional packet processing, packets using different congestion control algorithms are mixed in one buffer as shown in Figure1 (a). The mixing of CUBIC and BBR packets in the same buffer results in unequal throughput depending on the buffer size. In the proposed method, different buffers are provided for each congestion control algorithm at the relay node as shown in Figure1 (b) to avoid contention in the buffer, thereby alleviating the unfairness among the congestion control algorithms.

Figure2 shows an overview of the proposed method. The assumption is that the source host adds a flag indicating the congestion control algorithm to the option field of the TCP header for packets to be sent. The switch that relays the packets has a programmable data plane and multiple transmission buffers. Upon receiving a packet sent by the source host, the switch analyzes the TCP header using the programmable data plane to determine the congestion control algorithm used by the packet sender. The packets are then stored in the corresponding send buffers for each congestion control algorithm used.

When sending packets, the buffer containing CUBIC packets and the buffer containing BBR packets each count the number of packets sent. The following three packet scheduling schemes are used to retrieve packets from the buffers.

1) Fair Queueing (FQ)
   Check the buffer with the smallest number of packets to be transmitted, and if there are any packets to be transmitted, transmit them. If there are no packets, the next buffer with the lowest number of packets is checked, and if there are packets, they are sent. This process is repeated for as many buffers as there are buffers.
2) Strict Fair Queueing (SFQ)
   Send packets from each buffer in turn so that the number of packets to be sent is always equal. If there are no packets in the buffer, it waits for packets to arrive.
3) Fair Queueing with Holding Time (FQ-HT)
   Checks for packets from the buffer with the fewest number of packets to be transmitted and transmits if there are any packets. If there are no packets, it waits for a certain period of time for packets to arrive. If no packet arrives within a certain period of time, the next buffer is checked, and if there is a packet, it is transmitted. This process is repeated for as many buffers as there are buffers.

These packet scheduling operations are repeated for each packet sent. This implementation improves fairness among CUBIC and BBR with different transmission control methods.

IV. Performance evaluation by emulation

This section outlines the performance evaluation experiments of the proposed method using emulation, and describes the experimental results and discussion.

Authorized licensed use limited to: INDIAN INSTITUTE OF INFORMATION TECHNOLOGY. Downloaded on January 31,2026 at 16:35:05 UTC from IEEE Xplore. Restrictions apply.

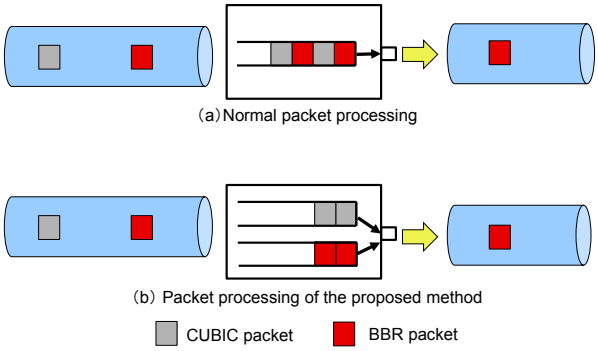


Fig. 1: Packet Processing

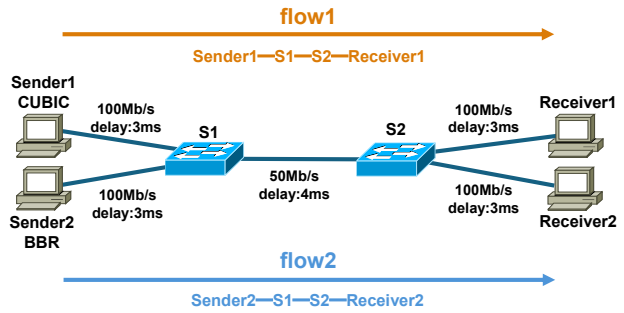


Fig. 3: Experimental Environment

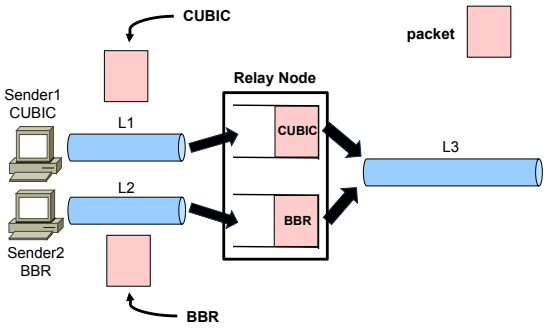


Fig. 2: Proposed Method

A. Summary of Performance Evaluation Experiments

To evaluate the throughput fairness of our proposed method, we perform emulation experiments and performance evaluation. We configured the experimental topology shown in Figure3 using the network emulator mininet [13]. The relay switch is a software switch BMv2 [14] that can run the data plane programming language P4. The bottleneck link is between S1 and S2, with delays of 3 ms between Sender1 and Sender2 and S1, 3 ms between S2 and Receiver1 and Sender2, and 4 ms between S1 and S2, so that the RTT between Sender and Receiver is 20 ms. A TCP flow is generated from each Sender to Receiver for 100 seconds using iperf. We measured the average (20s - 100s) of the interval throughput to remove the effects of connection and disconnection times and to measure the steady-state throughput. The buffer size of the bottleneck link was varied from 1 BDP, 8 BDP, and 32 BDP, and measurements were taken three times each. In this verification, we compare the results using Fair Queueing, Strict Fair Queueing, and Fair Queueing with Holding Time described in the previous section, as well as a method in which all TCP flows use the same buffer (existing method) without using the proposed method. The waiting time in the case of no packets in the buffer is set to 240 µs, which is the transmission time of one packet in a 50 Mb/s bottleneck link. Jain's fairness index [15], which is expressed by the following equation, is used as the fairness evaluation index.

$$FairnessIndex = \frac{(\sum_{i=1}^{n} x_i)^2}{(n \times \sum_{i=1}^{n} x_i^2)'} \tag{1}$$

where n is the number of senders and x_i is the throughput of the i-th flow.

B. Experimental Results and Discussion

1) 1 BDP: The time variation of throughput for each packet scheduling is shown in Figures 4 (a) to 4 (d), the average throughput in Figure 5, and the fairness index in Figure 6 when the buffer size of the bottleneck link is 1 BDP. Figures 4a and 5 show that BBR dominates the existing method when the buffer size of the bottleneck link is small. When the buffer size of the bottleneck link is small, packet loss due to buffer overflow occurs frequently. Packet loss causes CUBIC to reduce its transmission rate. On the other hand, BBR does not change the sending rate because BDP changes the sending rate regardless of packet loss. Therefore, BBR packets occupy the buffer and BBR becomes dominant.

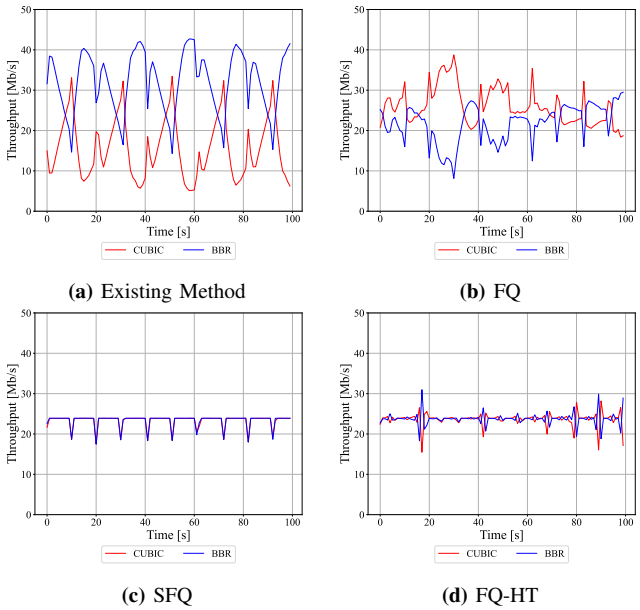
Figures 4 (b) and 5 show that the difference in throughput between Fair Queueing and the existing method is smaller. By separating buffers for each congestion control algorithm, packets using BBR no longer occupy the buffers, and the frequency of packet loss in CUBIC has decreased, which is thought to have improved throughput fairness.

Figures 4 (c), 4 (d) and 5 show that the throughputs of CUBIC and BBR are comparable for Strict Fair Queueing and Fair Queueing with Holding Time. Compared to Strict Fair Queueing, which controls transmission so that the number of packets sent by CUBIC and BBR are always one to one, Fair Queueing with Holding Time, which checks the next buffer if no packet arrives during the waiting time, achieves equal fairness. From this result, it can be said that the throughput between CUBIC and BBR can be made fair by setting a waiting time equal to the time required to send a single packet.

Figure 6 shows that all the proposed methods improve fairness compared to the existing methods. Strict Fair Queueing and Fair Queueing with Holding Time achieve particularly high fairness.

2) 8 BDP: The time variation of throughput for each packet scheduling is shown in Figures 7 (a) to 7 (d), the average

Authorized licensed use limited to: INDIAN INSTITUTE OF INFORMATION TECHNOLOGY. Downloaded on January 31,2026 at 16:35:05 UTC from IEEE Xplore. Restrictions apply.



(a) Existing Method **(b)** FQ

(c) SFQ **(d)** FQ-HT

Fig. 4: Time variation of throughput at 1BDP

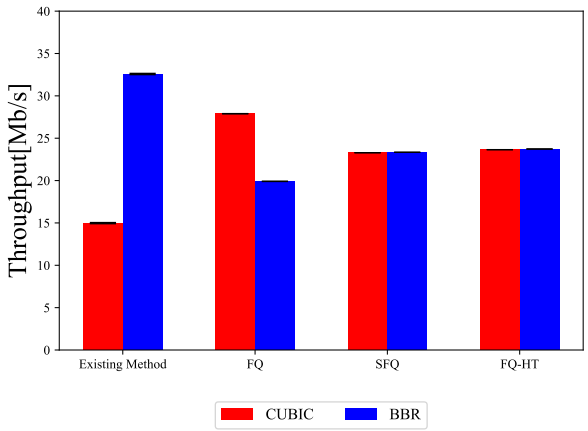


Fig. 5: Throughput (1BDP)

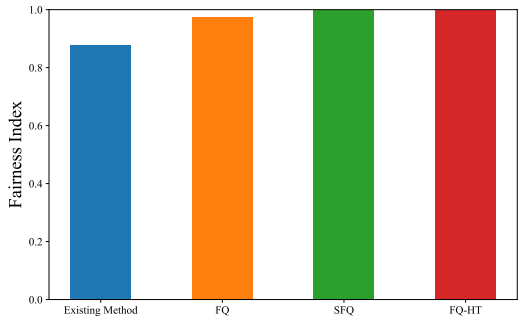
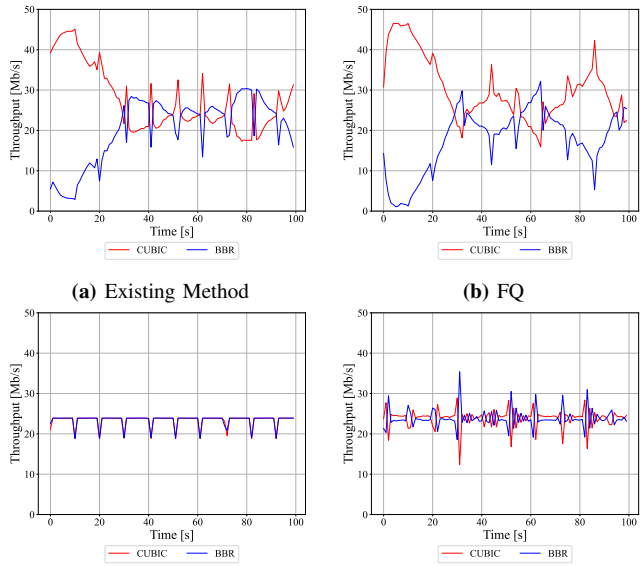


Fig. 6: Jain's fairness index (1BDP)



(a) Existing Method **(b)** FQ

(c) SFQ **(d)** FQ-HT

Fig. 7: Time variation of throughput at 8BDP

throughput in Figure 8, and the fairness index in Figure 9 when the buffer size of the bottleneck link is 8 BDP.

Figures 7 (a) and 8 show that the existing method alternates between CUBIC-dominated and BBR-dominated segments, and that the difference in throughput between CUBIC and BBR is small.

Figures 7 (b) and 8 show that the difference in throughput for fair queuing is slightly larger than for the conventional method. Comparing the results of Fair Queueing with those of CUBIC and BBR in Figures 5 and 11, the ratios of CUBIC and BBR throughputs are similar, indicating that Fair Queueing provides similar transmission control results when the buffer size is between 1 BDP and 32 BDP. The reason for this result is thought to be that BBR performs transmission control so that packets do not accumulate in the buffer, and therefore there are times when no packets exist in the buffer.

Figures 7 (c), 7 (d) and 8 show that the throughputs of CUBIC and BBR are comparable for Strict Fair Queueing and Fair Queueing with Holding Time. This result indicates that it is possible to make the throughput between CUBIC and BBR fair in 8BDP as in the case of 1BDP by setting the waiting time required to send a single packet.

Figure 9 shows that the existing methods, Strict Fair Queueing and Fair Queueing with Holding Time have high fairness, while Fair Queueing has low fairness.

3) 32 BDP: The time variation of throughput for each packet scheduling is shown in Figures 10 (a) to 10 (d), the average throughput in Figure 11, and the fairness index in Figure 12 when the buffer size of the bottleneck link is 32 BDP. Figures 10 (a) and 11 show that CUBIC dominates the existing method when the buffer size of the bottleneck link is large. When the buffer size of the bottleneck link is large, CUBIC increases the transmission rate until packet loss occurs. On the other hand, BBR maintains a constant sending rate because the sending rate is determined by the BDP. Therefore, packets using CUBIC occupy the buffer, resulting in unfair throughput.

Figures 10 (b) and 11 show that the difference in throughput

Authorized licensed use limited to: INDIAN INSTITUTE OF INFORMATION TECHNOLOGY. Downloaded on January 31,2026 at 16:35:05 UTC from IEEE Xplore. Restrictions apply.

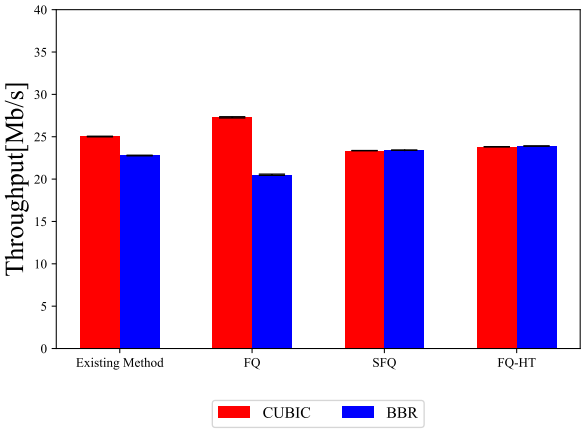


Fig. 8: Throughput (8BDP)

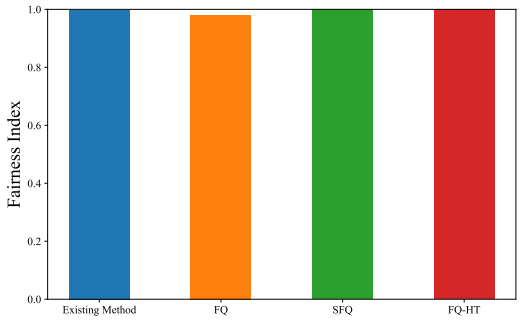


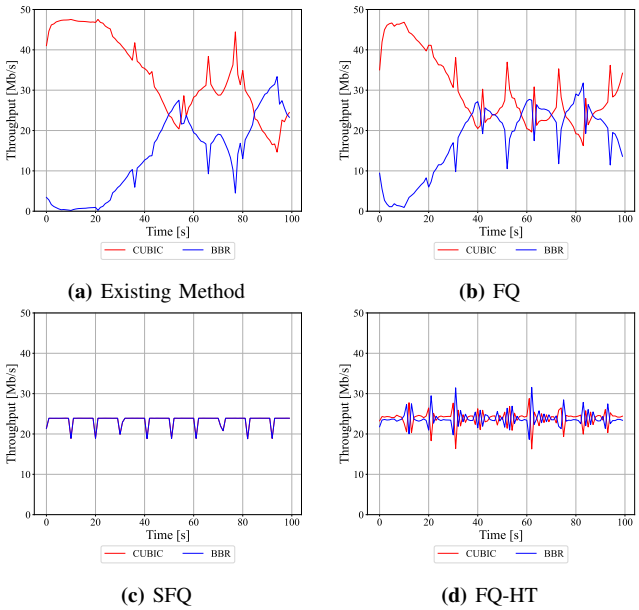
Fig. 9: Jain's fairness index (8BDP)

between Fair Queueing and the existing method is smaller. By separating buffers for each congestion control algorithm, it is considered that throughput fairness is improved because packets using CUBIC no longer occupy buffers.

Figures 10 (c), 10 (d) and 11 show that the throughputs of CUBIC and BBR are comparable for Strict Fair Queueing and Fair Queueing with Holding Time. This result indicates that it is possible to make the throughput fair between CUBIC and BBR by adding a waiting time required for one packet transmission even in 32BDP.

Figure 12 shows that all the proposed methods improve fairness compared to the existing methods. Strict Fair Queueing and Fair Queueing with Holding Time achieve high fairness as in the case of 1BDP.

4) Total throughput for each buffer size: Figures 13, 14, and 15 show the total throughput of CUBIC and BBR for buffer sizes of 1, 8, and 32 BDP on the bottleneck link. The total throughput of Strict Fair Queueing is approximately 1 Mb/s lower than that of CUBIC for all buffer sizes. Strict Fair Queueing waits for packets to arrive at a buffer with a small number of packets to be sent. Since BBR performs transmission control so that packets do not accumulate in the buffer, it is expected that there are times when no packets exist in the buffer. Figures 4 (c), 7 (c), and 10 (c) show that Strict Fair Queueing decreases throughput approximately once every 10 seconds. This is due to ProbeRTT, one of the BBR



(a) Existing Method **(b)** FQ

(c) SFQ **(d)** FQ-HT

Fig. 10: Time variation of throughput at 32BDP

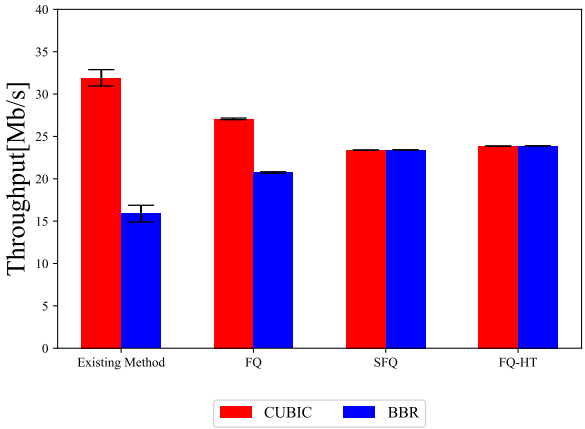


Fig. 11: Throughput (32BDP)

state transitions. ProbeRTT is a phase in which buffers are emptied to measure the minimum RTT without queuing delay. This is thought to reduce the total throughput. In contrast, Fair Queueing with Holding Time checks for the next buffer after 240μs, the time required to send one packet. Therefore, compared to Strict Fair Queueing, Fair Queueing with Holding Time does not have a significant impact on total throughput because it takes less time to wait for packets to arrive.

V. Conclusion

In this study, we focused on the issue of unequal throughput when CUBIC and BBR contend with each other at the bottleneck link. As a method to improve the problem, we proposed a method to provide a buffer for each congestion control algorithm using the data plane programming language P4. The effectiveness of the proposed method was evaluated by changing the buffer size of the bottleneck link and packet

Authorized licensed use limited to: INDIAN INSTITUTE OF INFORMATION TECHNOLOGY. Downloaded on January 31,2026 at 16:35:05 UTC from IEEE Xplore. Restrictions apply.

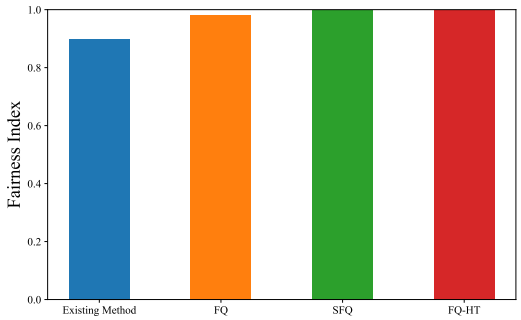


Fig. 12: Jain's fairness index (32BDP)

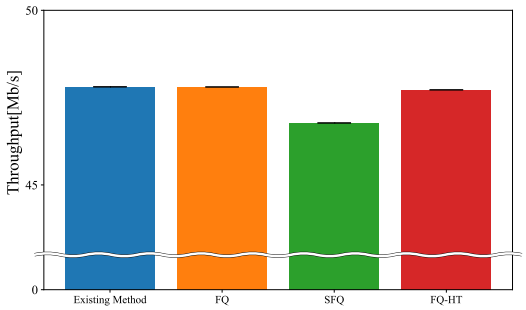


Fig. 14: Total Throughput (8BDP)

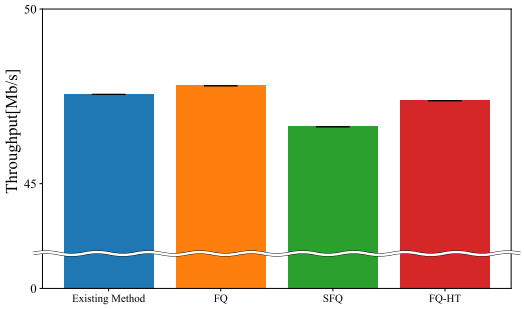


Fig. 13: Total Throughput (1BDP)

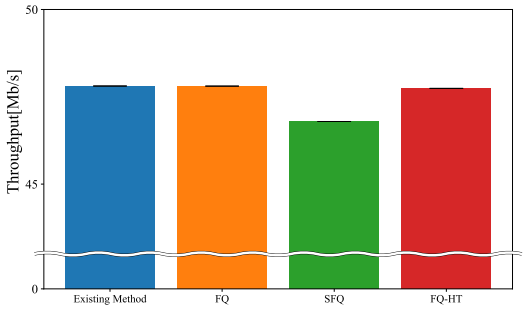


Fig. 15: Total Throughput (32BDP)

scheduling at the relay node through emulation. The performance evaluation results showed that fairness was improved for both Strict Fair Queueing and Fair Queueing with Holding Time, regardless of the buffer size. The total throughput of Strict Fair Queueing decreased by approximately 1 Mb/s. The Fair Queueing with Holding Time improved the total throughput by approximately 1 Mb/s. On the other hand, Fair Queueing with Holding Time can improve fairness while maintaining total throughput by setting an appropriate waiting time.

In the future, we will investigate the optimal waiting time for Fair Queueing with Holding Time when the network configuration is changed and verify the use of congestion control algorithms other than CUBIC and BBR.

References

[1] S. Ha, I. Rhee, and L. Xu, "CUBIC: A new TCP-friendly high-speed TCP variant," ACM SIGOPS oper. syst. rev., vol. 42, no. 5, pp. 64–74, July 2008.

[2] Neal Cardwell, Yuchung Cheng, C Stephen Gunn, Soheil Hassas Yeganeh, and Van Jacobson, "BBR: Congestion-Based Congestion Control: Measuring bottleneck bandwidth and round-trip propagation time," Queue, 14 (5) : 20-53, 2016,

[3] A. Mishra, X. Sun, A. Jain, S. Pande, R. Joshi, and B. Leong, "The Great Internet TCP Congestion Control Census," ACM Measurement and Analysis of Computing Systems, vol.3, Issue 3, no.45, pp.1-24, December 2019.

[4] R.Gail, et al., "Aninvarientpropertyofcomputernetworkpower," IEEEInternationalConferenceonCommunications, 63.1.1-63.1.5, 1981

[5] Yeong-Jun Song, Geon-Hwan Kim, and You-Ze Cho, "Improvement of Cyclic Performance Variation between TCP BBR and CUBIC," IEEE Asia-Pacific Conference on Communications(APCC), November 2019

[6] Natuki Katumata, and Saneyasu YANAGUCHI, "TCP Fairness Improvement by Congestion Control from User Space," International Conference on Engineering Technology for Communications, December 2021.

[7] Yeong-Jun Song, Geon-Hwan Kim, and You-Ze Cho, "Congestion Window Scaling Method for Inter-protocol Fairness of BBR," IEEE Anual Consumer Communications and Networking Conference(CCNC), March 2020

[8] Yeong-Jun Song et. al., "Understanding of BBRv2: Evaluation and Comparison With BBR v1 Congestion Control" IEEE ACCESS, pp.37131-37145, Feb. 2021

[9] S. Ishikura and M. Yamamoto, "BAR : BBR with Adjusting RTprop for Inter-Protocol Fairness with CUBIC TCP," 2023 IEEE 29th International Symposium on Local and Metropolitan Area Networks (LANMAN),pp. 1-6, July 2023

[10] Ranysha Ware et. al., "ModelingBBR's Interactioins with Loss-Based Congestion Control," IMC Proceedings of the Internet Measurment Conference, pp.137-143, Oct. 2019

[11] Satoshi Utsumi et. al., "Improving Inter-Protocol Fairness Based on Estimated Behavior of Competing Flows," IFIP Networking Conference, 2022

[12] P. Bosshart, D. Daly, G. Gibb, M. Izzard, N. McKeown, J. Rexford, C. Schlesinger, D. Talayco, A. Vahdat, G. Varghese, and D. Walker, "P4: Programming Protocol-Independent Packet Processors," ACM SIGCOM Computer Communication Review, vol44, no. 3, pp. 87-95, 2014.

[13] "Mininet: An Instant Virtual Network on your Laptop (or other PC)," [Online]. Avaliable: https://mininet.org, Accessed on : May 7, 2024

[14] "BEHAVIORAL MODEL (bmv2)," [Online]. Avaliable: https://github.com/p4lang/behavioral-model, Accessed on : May. 7, 2024

[15] K.R.Jain et. al, "A quantitative measure of fairness and discrimination," Eastern Research Laboratory, Digital Equipment Corporation, Hudson,MA, vol.21, Sep. 1984.

Authorized licensed use limited to: INDIAN INSTITUTE OF INFORMATION TECHNOLOGY. Downloaded on January 31,2026 at 16:35:05 UTC from IEEE Xplore. Restrictions apply.